university of groningen

Quantitative Text Analysis – Essex Summer School

Principles of QTA. Cleaning text. Preprocessing text

dr. Martijn Schoonvelde

University of Groningen

Today's class

- Day 1 carry-over: preparing a corpus and importing text into R
  - file formats, APIs, web scraping
- Principles of automated text analysis (Grimmer, Roberts, and Stewart, 2022): task dependence and validation
- Clean text in R: string operations and regular expressions with the **stringr** library
- Tokenization, preprocessing, and document-feature matrices with **quanteda**
- Define richer features: n-grams, collocations, and POS-informed features

Principles of Automated Text Analysis (Grimmer, Roberts & Stewart 2022)

These principles represent an agnostic approach to text analysis (as opposed to a structural approach)

1. Social science theories and domain knowledge are essential for research design
    - E.g., when examining parliamentary speech, we need to know how a bill becomes a law in a country.
2. QTA does not replace humans – it augments them
    - For example, QTA may help us categorise our documents or find key documents
3. We have to separate the discovery phase from the testing phase
4. The best method depends on the task
5. Validations are essential and depend on the theory and the task

Developing a corpus: sources of Bias (Grimmer, Roberts and Stewart, 2022)

1. Resource bias – texts often better reflect populations with more resources to produce, record and store documents
2. Incentive bias – strategic behavior can drive the production and retention of documents
   - E.g., skydive pictures on Instagram; hot takes on *X*
3. Medium bias – medium in which text is produced may constrain its content
   - E.g., more polite political discussions on Twitter after doubling of character limit (Jaidka *et al.*, 2019). See also algorithmic drift (Salganik, 2018)
4. Retrieval bias – our methods to sample documents may have biases baked into them
   - E.g., searching for newspaper articles about 'the economy' using 'economy' as a keyword

Where to find pre-existing corpora

- Repositories such as Lexis Nexis or Factiva (newspaper data)
- Existing text datasets. For example:
  - Parliamentary corpora: House of Commons speeches; Parlspeech (Rauh *et al.*); ParlEE (Silvester *et al.*)
  - Political speech corpora such as EUSpeech (Schumacher *et al.*)
  - UN General Debate Corpus (Jankin *et al.*)
  - DICEU (Wratil & Hobolt)
  - ECB speeches: https://www.ecb.europa.eu/press/key/html/downloads.en.html
  - Party manifestos: https://manifesto-project.wzb.eu/
  - A general repository of political datasets: https://github.com/erikgahner/PolData
- Replication data folders

Getting data from the web

Route	Advantages	Challenges / tools
APIs	Structured access to data made available by a platform	Authentication, rate limits, pagination, JSON wrangling
Web scraping	Useful when no API exists and the relevant text is visible on a page	Fragile page structure, terms of use, HTML/XML cleaning

- LLMs and coding agents can make these tasks easier: reading API documentation, drafting requests, debugging selectors, parsing JSON, HTML, or XML, and turning messy output into data frames
- They help with implementation, but not with research design choices

Preparing texts for analysis

Once you've selected your texts, they need to be prepared for analysis. The same preparation logic applies to any corpus:

1. **Import the texts:** Load your documents into R.
2. **Clean the texts:** Remove unwanted content such as metadata, HTML tags, punctuation, and numbers. This helps focus the analysis on meaningful words.
3. **Preprocess the texts:** Standardize the text by applying steps such as lowercasing, stemming or lemmatization, and removing stopwords. The goal is to simplify the text while preserving meaning.

Importing text into R (1)

Format / source	Useful R packages
`.txt`, `.csv`, `.tsv`	`readr`, `data.table`
`.json`	`jsonlite`
`.html`, `.xml`	`rvest`, `xml2`
`.pdf`	`pdftools`
`.docx`	`readtext`
APIs and web data	`httr2`, `jsonlite`, `rvest`

- LLMs and coding agents can help draft import code, parse nested JSON, debug selectors, and translate examples across packages
- But always inspect the raw file and imported object: encoding, missing values, duplicated rows, metadata, and document boundaries

Importing text into R (2)

Challenges with character encoding:

- When importing text – especially in non-English languages – you may encounter issues with character encoding.
- Text is stored using encoding schemes like UTF-8, UTF-16, or ASCII. If R guesses the wrong encoding, you may see strange symbols or question marks.
- There is no one-size-fits-all solution; the correct encoding often depends on the original source and format.

Example: Specify encoding manually

```
text <- readLines("data/french_speech.txt", encoding = "UTF-8")
```

This tells R to read the file assuming it uses UTF-8 encoding (common for web text and modern documents).

More on encoding: http://kunststube.net/encoding/

A messy House of Commons snippet

What would you clean, preserve, or turn into features?

Field	Value
date	1997-01-20
speaker	William Hague
party	Conservative
agenda	Public Expenditure [Oral Answers To Questions > Wales]

[Interruption.] I can assure the hon. Gentleman that I shall be cheering for Wales in any rugby match against anybody, including England–although I might stay out of a cricket match between Glamorgan and Yorkshire. [. . .] If the hon. Gentleman honestly believes that we should swap our economic circumstances with those of Ireland, where unemployment is DRAMATICALLY higher than in Wales, and when Wales is benefiting from record inward investment!!! he must be out of his mind.

Cleaning Text with String Operations

- You can clean and manipulate text using **base R** functions like `gsub()`, `tolower()`, and `nchar()` – no extra packages needed.
- For more readable and powerful string handling, use packages like:
  - **stringr** (part of the Tidyverse): Consistent syntax and naming.
  - **stringi**: Fast and supports complex operations.
- **Regular Expressions (regex)** allow you to match patterns in text:
  - Powerful for removing or extracting specific content (e.g., email addresses, punctuation, tags).
  - Often requires trial and error, especially for complex patterns.

The `stringr` package

Key features of `stringr`:

- All functions begin with `str_` for consistency.
- The first argument is always a character vector.
- Most functions support **regular expressions** for flexible pattern matching.
- Helpful guide: **stringr** cheat sheet – https://github.com/rstudio/cheatsheets/blob/master/strings.pdf

Examples:

```r
agenda <- c(
  "Export Trade",
  "British Prisoners of War, Far East",
  "Business of the House"
)

str_length(agenda)
# [1] 12 34 21

str_count(agenda, "War")
# [1] 0 1 0
```

stringr example

```
agenda <- c(
  "  Export Trade [Commons] -- vol. 407  ",
  "BRITISH PRISONERS OF WAR, FAR EAST!!!",
  "Business   of the House @ 3.30pm"
)

# Remove bracketed context
agenda <- str_remove(agenda, "\\s*\\[.*\\]")
print(agenda)

# [1] "  Export Trade -- vol. 407"
# [2] "BRITISH PRISONERS OF WAR, FAR EAST!!!"
# [3] "Business   of the House @ 3.30pm"
```

stringr example

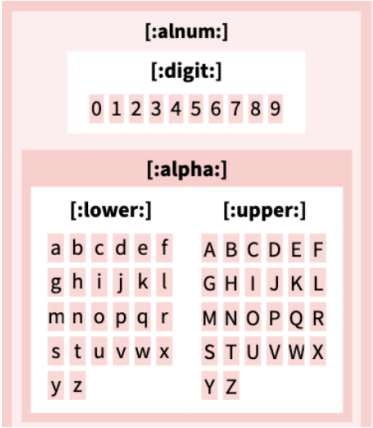
Transform to lower case

```
agenda <- str_to_lower(agenda)
print(agenda)

# [1] "  export trade -- vol. 407"
# [2] "british prisoners of war, far east!!!"
# [3] "business   of the house @ 3.30pm"
```

stringr example

Remove everything but letters or numbers

```
agenda <- str_replace_all(agenda, "[^[:alnum:]]", " ")
print(agenda)

# [1] "  export trade    vol  407"
# [2] "british prisoners of war  far east   "
# [3] "business   of the house   3 30pm"
```



Source: Posit stringr cheat sheet

stringr example

Remove multiple white spaces, as well as surrounding white spaces

```
agenda <- str_squish(agenda)
print(agenda)

# [1] "export trade vol 407"
# [2] "british prisoners of war far east"
# [3] "business of the house 3 30pm"
```

House of Commons speech contribution

> *Yes, it is, because I agree with the hon. and learned Gentleman that it is the strength of our deterrent that counts in a moment like this. I am very proud of our armed forces – but now is a time to ask more of them and to step up.*

Keir Starmer, Labour. Delivered during the Defence and Security debate, House of Commons, 25 February 2025.

Image: House of Commons / Wikimedia Commons, CC BY 3.0

Bag of Words Representation

- The **Bag of Words (BoW)** model represents text numerically
- Also known as a **Document-Term Matrix (DTM)** or **Document-Feature Matrix (DFM)**
- Structure:
  - rows = documents
  - columns = words or features
  - cells = counts, presence/absence, or weights

> "... strength of our deterrent ...
> proud of our armed forces ..."

⇓

Unigram counts

doc	strength	of	our	deterrent	proud	armed	forces
Starmer	1	2	2	1	1	1	1

Binary features

doc	strength	of	our	deterrent	proud	armed	forces
Starmer	T	T	T	T	T	T	T

Variation: TF-IDF

doc	strength	of	our	deterrent	proud	armed	forces
Starmer	.06	.01	.02	.42	.18	.32	.50

Implications of the Bag of Words Model

What BoW keeps

- A transparent numerical representation
- Counts, weights, or binary indicators
- Can serve as input for:
  - dictionary analysis
  - topic models
  - supervised classification

```
speech -> tokens -> counts
```

What BoW loses

issue	example
word order	`not good` vs. `good`
ambiguity	`bank` = finance or river?
context	sarcasm, negation, framing
syntax	who did what to whom?

Partial BoW fixes: n-grams, phrase detection

Bag of Words Model Pipeline

1. **Choose the unit of analysis:** speech, sentence, paragraph, speaker-day, party-year
2. **Tokenize:** split text into words, phrases, or n-grams
3. **Reduce complexity:** lowercase, remove punctuation/stopwords, stem or lemmatize, trim features
4. **Build the Document-Feature Matrix (DFM):** rows = documents, columns = features
5. **Inspect and validate:** check top features, sparsity, and sensitivity to preprocessing choices

Tokenization

Tokenization is the process of dividing text into individual units – usually words or phrases.

- Each unit is called a token.
- Tokens that are the same (e.g., repeated words) belong to the same type.
- The full set of unique types in a corpus is called the vocabulary.

Example:

```
Text: "Health services need health workers."

Tokens:      ["Health", "services", "need", "health", "workers", "."]
Types:       {"Health", "services", "need", "health", "workers", "."}
             (before lowercasing: "Health" and "health" are different)
Vocabulary:  {"health", "services", "need", "workers"}
             (after lowercasing and removing punctuation)
```

Tokenizers

BoW models

- Often uses word tokens
- Tokenization choices are usually visible and researcher-controlled
- Preprocessing often happens before modeling

More advanced models

- LLM tokens are usually subwords, not words
- Common families:
  - **Subword tokens:** frequent character sequences
  - **Character tokens:** individual characters
  - **Byte tokens:** raw bytes of text

Tokenization is not just a technical preprocessing step but it shapes what the model "sees," how much text fits, what it costs, and how robust the analysis is.

Word Tokenization (1)

```
hoc_speech <- corpus(starmer_speech)
hoc_tokens <- tokens(hoc_speech)
as.list(hoc_tokens)[[1]][1:45]

 [1] "Yes"       ","         "it"        "is"        ","
 [6] "because"   "I"         "agree"     "with"      "the"
[11] "hon"       "."         "and"       "learned"   "Gentleman"
[16] "that"      "it"        "is"        "the"       "strength"
[21] "of"        "our"       "deterrent" "that"      "counts"
[26] "in"        "a"         "moment"    "like"      "this"
[31] "."         "I"         "am"        "very"      "proud"
[36] "of"        "our"       "armed"     "forces"    "-"
[41] "-"         "but"       "now"       "is"        "a"
```

NB everything that is not a white space is tokenized

Word Tokenization (2)

The simplest way to tokenize is to divide into unigrams, but sometimes we lose important information (e.g., United Kingdom, armed forces). This is where multiword expressions come in.

```
hoc_tokens <- tokens_compound(
  hoc_tokens,
  phrase("armed forces"),
  concatenator = "_"
)
as.list(hoc_tokens)[[1]][35:45]
```

```
 [1] "proud"        "of"           "our"          "armed_forces"
 [5] "-"            "-"            "but"          "now"
 [9] "is"           "a"            "time"
```

Beyond Unigrams

The default BoW representation usually starts from unigrams: individual word features.

But we often want richer feature definitions:

Feature Type	Example	Why Use It?
unigram	`security`	simple, transparent baseline
bigram	`armed_forces`	preserves short phrases
trigram	`prime_minister_said`	preserves longer expressions
collocation	`northern_ireland`	identifies statistically associated terms
POS-informed feature	nouns or adjectives only	targets specific linguistic functions

Key point: feature design is a substantive measurement choice.

Bigrams and Trigrams

N-grams represent consecutive word sequences.

- **Bigram:** two-word sequence
- **Trigram:** three-word sequence
- Useful for:
  - negation: `not_good`
  - named entities: `prime_minister`
  - policy phrases: `armed_forces`
- They increase the number of features quickly and with lots of zeroes (very sparse representation)

```
hoc_bigrams <- tokens(
  hoc_speech,
  remove_punct = TRUE
) |>
  tokens_tolower() |>
  tokens_ngrams(n = 2)

as.list(hoc_bigrams)[[1]][18:24]

# [1] "strength_of" "of_our" "our_deterrent"
# [4] "deterrent_that" "that_counts"
# [6] "counts_in" "in_a"
```

Collocations

Collocations are word sequences that occur more often than expected given the frequency of the individual words.

- They are useful when a phrase means more than the sum of its parts:
  - `prime_minister`
  - `northern_ireland`
  - `armed_forces`
- Association measures compare observed co-occurrence with expected co-occurrence
- Rare pairs can look overly strong, so use frequency thresholds

Collocations in Quanteda

```
hoc_collocations <- textstat_collocations(
  hoc_tokens_dfm,
  size = 2,
  min_count = 10
)

hoc_collocations |>
  arrange(desc(lambda)) |>
  head(8)

#              collocation count lambda
# 1          prime minister   ...
# 2        northern ireland   ...
# 3            armed forces   ...
```

The exact results depend on preprocessing, vocabulary trimming, and the sample. This is why collocations should be inspected before being compounded into features.

Remove punctuation

```
hoc_tokens <- tokens(hoc_speech, remove_punct = TRUE)
hoc_tokens <- tokens_compound(hoc_tokens, phrase("armed forces"))

as.list(hoc_tokens)[[1]][1:34]

#  [1] "Yes"       "it"        "is"        "because"   "I"
#  [6] "agree"     "with"      "the"       "hon"       "and"
# [11] "learned"   "Gentleman" "that"      "it"        "is"
# [16] "the"       "strength"  "of"        "our"       "deterrent"
# [21] "that"      "counts"    "in"        "a"         "moment"
# [26] "like"      "this"      "I"         "am"        "very"
# [31] "proud"     "of"        "our"       "armed_forces"
```

NB everything that is not a white space is tokenized

Remove stopwords

```
hoc_tokens <- tokens_select(
  hoc_tokens,
  pattern = stopwords("en"),
  selection = "remove"
)

as.list(hoc_tokens)[[1]]

#  [1] "Yes"          "agree"        "hon"          "learned"
#  [5] "Gentleman"    "strength"     "deterrent"    "counts"
#  [9] "moment"       "like"         "proud"        "armed_forces"
# [13] "now"          "time"         "ask"          "step"
```

Stemming

- Stemming: algorithmic conversion of inflected forms of words into their root forms
- Fast but not perfect:
  - Unrelated words may be grouped together; related words may not be grouped together
  - Stems may not be words themselves – problematic if further analysis is based on dictionaries

```
hoc_tokens_stemmed <- tokens_wordstem(
  hoc_tokens,
  language = quanteda_options("language_stemmer")
)

as.list(hoc_tokens_stemmed)[[1]]

#  [1] "Yes"       "agre"      "hon"       "learn"     "Gentleman"
#  [6] "strength"  "deterr"    "count"     "moment"    "like"
# [11] "proud"     "arm"       "forc"      "now"       "time"
# [16] "ask"       "step"
```

Lemmatization

- Use of a dictionary to replace words with their root form
- Results are always words; neither does it group together unrelated words, nor does it miss to group together related words

```
words <- c("counts", "forces")
lemma <- c("count", "force")

hoc_tokens_lemmatized <- tokens_replace(
  hoc_tokens,
  words,
  lemma,
  valuetype = "fixed"
)

as.list(hoc_tokens_lemmatized)[[1]]

#  [1] "Yes"       "agree"     "hon"       "learned"   "Gentleman"
#  [6] "strength"  "deterrent" "count"     "moment"    "like"
# [11] "proud"     "armed"     "force"     "now"       "time"
# [16] "ask"       "step"
```

POS-Informed Features

Part-of-speech tagging adds grammatical information to tokens:

Token	Lemma	POS
strength	strength	NOUN
deterrent	deterrent	NOUN
proud	proud	ADJ
ask	ask	VERB

- POS tags can support feature design:
  - sentiment: adjectives and adverbs
  - policy objects: nouns and proper nouns
  - action frames: verbs
- We return to POS tagging, lemmatization, and dependency parsing in Day 8

Preprocessing data in languages other than English

Global linguistic diversity is enormous...

- Some 6,000–7,000 languages are spoken globally today
- Among other dimensions they vary in their script (e.g, Latin, Cyrillic, and Greek), their morphology (the system through which words get formed) and their syntax (how words are brought together to make sentence)

...but diversity in computational tools and methods much less so

- Keep in mind that preprocessing steps are language-specific!

dfm in quanteda

When we are happy with our features, we can create the document feature matrix using the dfm function

```
dfm(hoc_tokens)[, 1:10]

# Document-feature matrix of: 1 document, 10 features (0.00% sparse)

#        features
# docs    yes agree hon learned gentleman strength deterrent counts moment like
#   text1   1     1   1       1         1        1         1      1      1    1
```

dfm in quanteda

```
hoc_dfm <- hoc_corpus |>
  tokens(remove_punct = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords("en")) |>
  dfm()

dim(hoc_dfm)
# [1] 5000 30598

topfeatures(hoc_dfm, 8)
# hon government right member can people one friend
# 6569 3213 2780 2428 2423 2296 2288 2279
```

Filtering and Weighting

"the greatest signals from words are those in the goldilocks region – neither too rare nor too frequent" (Grimmer, Roberts and Stewart, p. 75)

- **Feature Selection:** Not all terms contribute equally to an analysis. It's often useful to exclude:
  - Very common words that appear in many documents.
  - Very rare words that appear in few documents.
- **Document Frequency Filtering:** Apply thresholds to filter terms based on how frequently they appear across documents, either as a count or proportion.
- **Weighting Methods:** Assign weights to terms to enhance relevance, using techniques like tf-idf (term frequency-inverse document frequency).

Understanding TF-IDF Weighting

- **Term Frequency (TF):** Measures how frequently a term occurs in a document. The more often a term appears in a document, the higher its importance within that document.
- **Inverse Document Frequency (IDF):** Measures the importance of the term across a set of documents. The more documents the term appears in, the less unique it is, thus lowering its IDF score.
- **TF-IDF:** Combines TF and IDF, resulting in a weight that increases with the number of times a term appears in a document but decreases with the frequency of the term across documents.
- **Intuition:** TF-IDF penalizes common terms while giving higher weight to terms that are unique to a particular document. This helps in distinguishing documents and understanding their specific context.